

# Teaching Exam

**Subject – Computer Science**

**Topic – Computer Organization &  
Architecture**

**Lecture No.- 02**



**By- Rahul Sir**

# Topics to be Covered

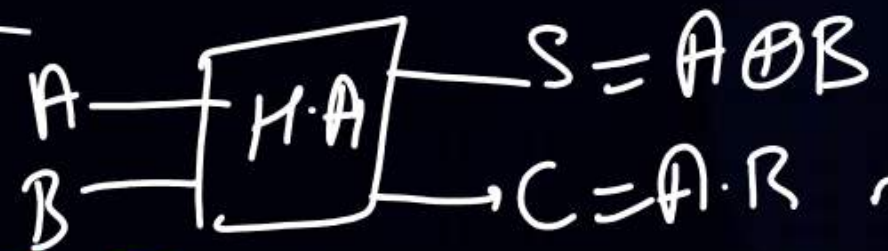
## Topic



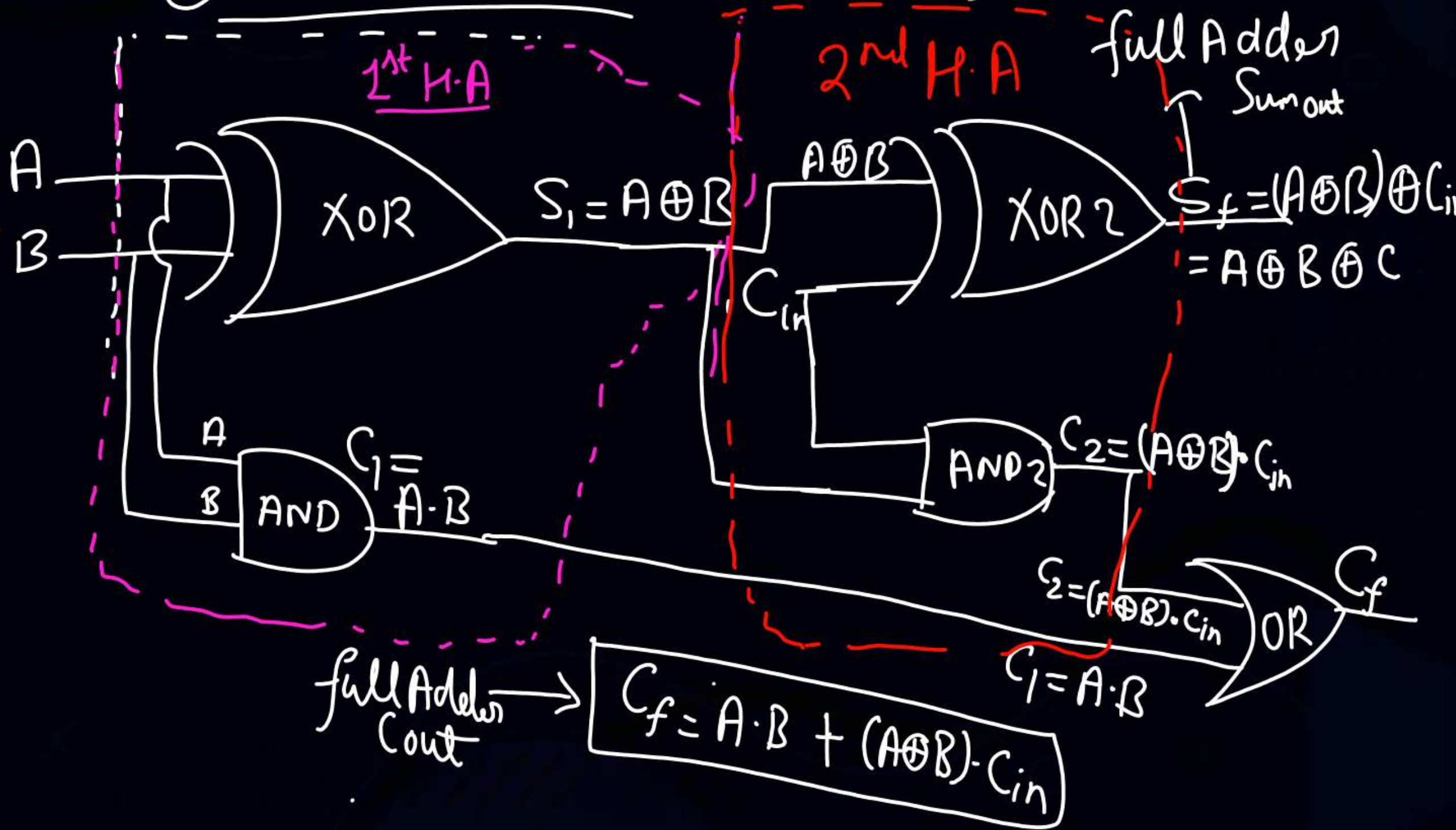
- 1:- Binary Subtractor ✓
- 2:- Half Subtractor ✓
- 3:- Full subtractor ✓
- 4:- Multiplexer ✓
- 5:- Types of Multiplexer
- 6:- 2x1 mux, 4x1 mux, 8x1 mux, 16x1 mux ✓
- 7:- 8x1 mux by using 2x1 mux
- 8:- 8x1 mux by using 4x1 mux – special case (Enabling System)

# Full Adder using Half Adder

## Circuit Diagram



~~2 H.A & 1 OR Gate~~  
~~X 2 H.A~~



Full Adder Cout



In digital electronics, a **subtractor** is a combinational logic circuit that can perform the subtraction of two number (binary numbers) and produce the difference between them. It is a combinational circuit that means its output depends on its present inputs only. Although, in practice, the subtraction of two binary number is accomplished by taking the 1's or 2's compliment of the subtrahend and adding it to the minuend.

In this way, the subtraction operation of binary numbers can be converted into simple addition operation which makes hardware construction simple and less expensive. There are two types of subtractors namely, **Half Subtractor** and **Full Subtractor**.



A **half-subtractor** is a combinational logic circuit that have two inputs and two outputs (i.e. difference and borrow). The half subtractor produces the difference between the two binary bits at the input and also produces a borrow output (if any). In the subtraction (A-B), A is called as **Minuend bit** and B is called as **Subtrahend bit**. The block diagram and logic circuit diagram of the half subtractor is shown in Figure-1.

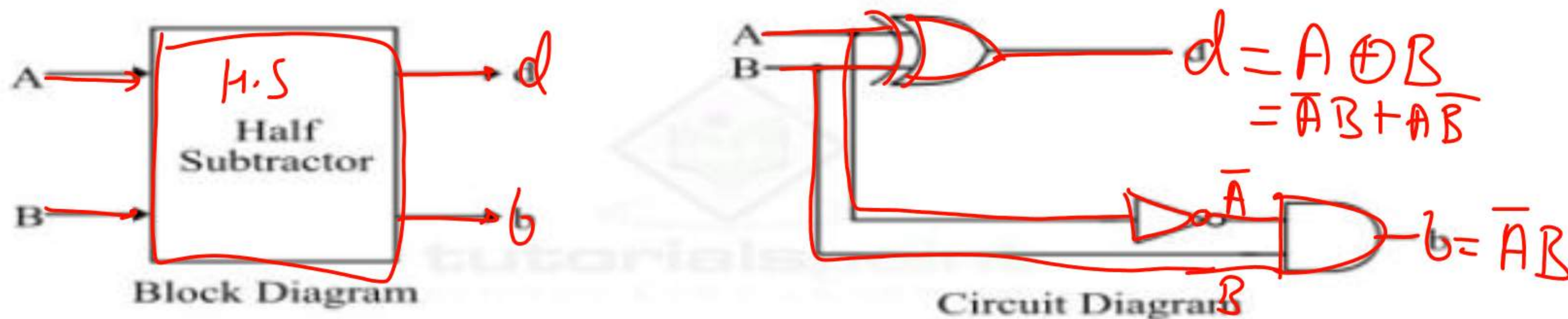


Figure 1 - Half Subtractor

Hence, from the logic circuit diagram, it is clear that a half subtractor can be realized using an XOR gate together with a NOT gate and an AND gate.

In the half subtractor as shown in figure-1, A and B are the inputs, d and b are the outputs. Where, d indicates the difference and b indicates the borrow output. The borrow output (b) is the signal that tells the next stage that a 1 has been borrowed.



## Operation of Half Subtractor

Now, let us understand the operation of the half subtractor circuit. Half subtractor performs its operation to find the difference of two binary digits according to the rules of binary subtraction, which are as follows –

The output borrow of  $b$  is zero (0) as long as the minuend bit ( $A$ ) is greater than or equal to the subtrahend bit ( $B$ ), i.e.  $A \geq B$ . The output borrow is a 1 when  $A = 0$  and  $B = 1$ .

From the logic circuit diagram of the half subtractor, it is clear that the difference bit ( $d$ ) is obtained by the XOR operation of the two inputs  $A$  and  $B$ , and the borrow bit is obtained by AND operation of the complement of the minuend ( $A'$ ) with the subtrahend ( $B$ ).



### Truth Table of Half Subtractor

The following is the truth table the half-subtractor

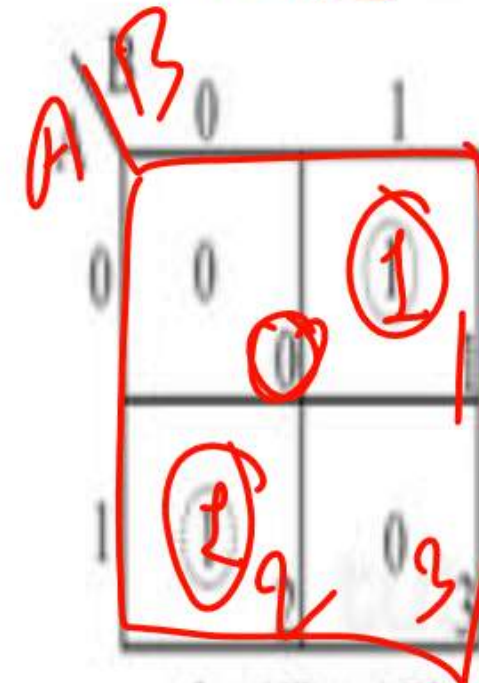
| Inputs |   | Outputs        |            |
|--------|---|----------------|------------|
| A      | B | D (Difference) | B (Borrow) |
| 0      | 0 | 0              | 0          |
| 0      | 1 | 1 ✓            | 1 ✓        |
| 1      | 0 | 1 ✓            | 0          |
| 1      | 1 | 0              | 0          |

### K-Map for Half Subtractor

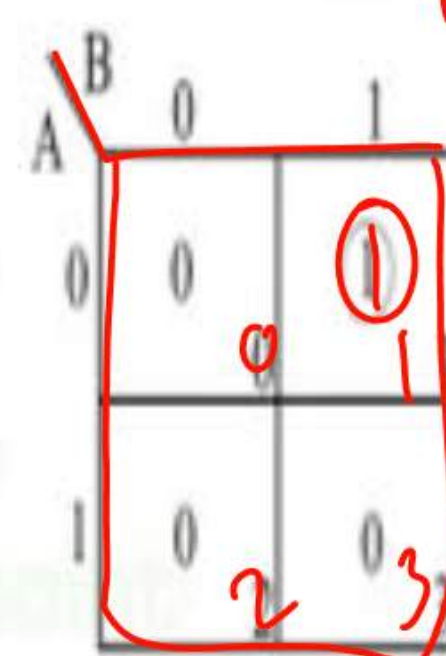
We can use the K-Map (or Karnaugh Map), a method for simplifying Boolean algebra, to determine equations of the difference bit (d) and the output borrow (b).

The K-Map simplification for half subtractor is shown in Figure-2.

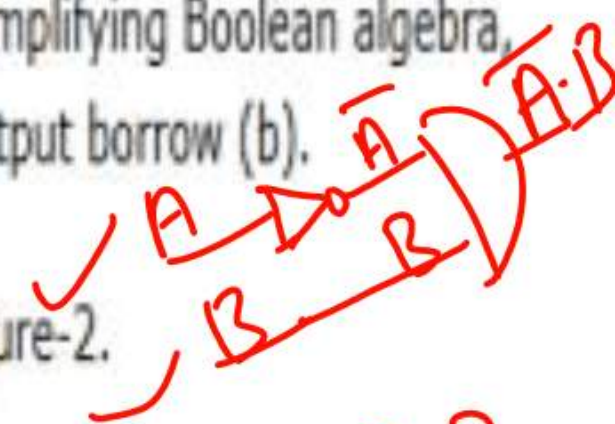
$$d = \bar{A}B + A\bar{B} = A \oplus B$$



$$d = A'B + AB'$$



$$b = \bar{A}B$$





## Characteristic Equation of Half Subtractor

The characteristic equations of the half subtractor, i.e. equations of the difference bit (d) and the output borrow bit (b) are obtained by following the rules of binary subtraction. These equations are given as follows –

The difference bit (d) of the half subtractor is given by XORing the two inputs A and B. Therefore,

$$\text{Difference, } d = A \oplus B = A'B + AB'$$

The borrow (b) of the half subtractor is the AND of A (compliment of A) and B. Therefore,

$$\text{Borrow, } b = A'B$$



## Applications of Half Subtractor

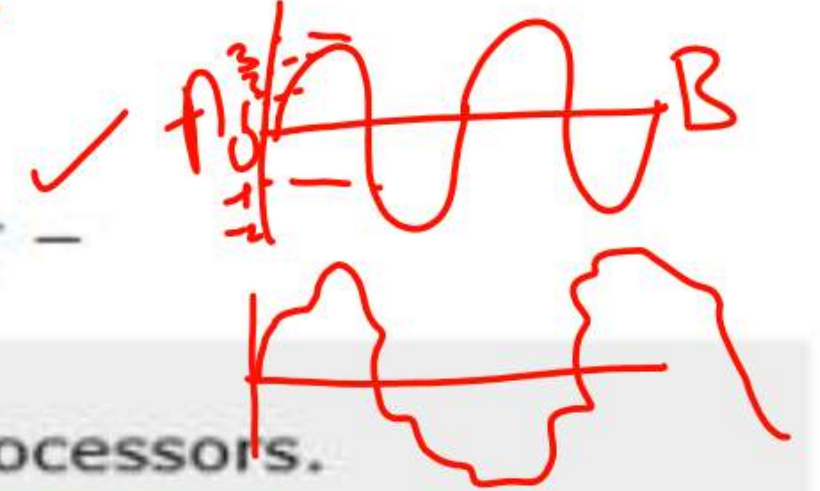
The following are some important applications of half subtractor –

- Half subtractor is used in ALU (Arithmetic Logic Unit) of processors.
- Half subtractor can also be used in amplifiers to compensate the sound distortion.
- It is also used to decrease the force of radio signals or audio signals.
- Half subtractor is also used to increase or decrease operators.

## Conclusion

From the above discussion, we can conclude that a half subtractor is a combinational logic circuit that can calculate the difference of two binary digits. A half subtractor can only be used to subtract the LSB (Least Significant Bit) of the subtrahend from the LSB of the minuend when one binary number is subtracted from another binary number.

HELLO

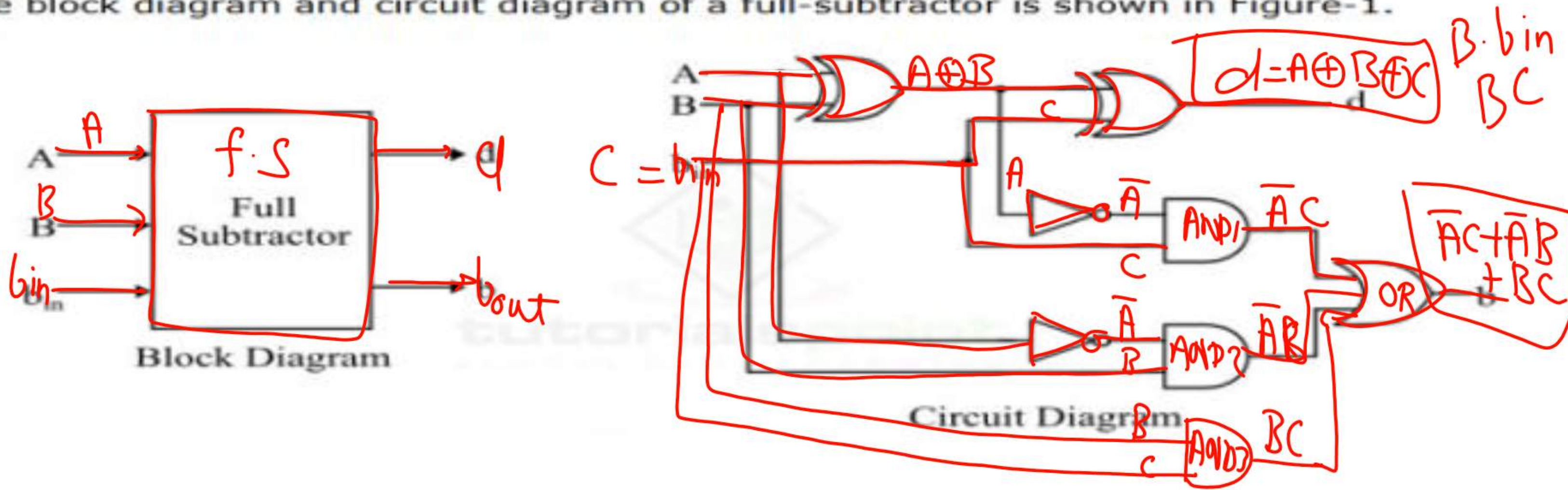




A **full-subtractor** is a combinational circuit that has three inputs A, B,  $b_{in}$  and two outputs d and b. Where, A is the minuend, B is subtrahend,  $b_{in}$  is borrow produced by the previous stage, d is the difference output and b is the borrow output.

As we know that the half-subtractor can only be used for subtraction of LSB (least significant bit) of binary numbers. If there is any borrow during the subtraction of the LSBs of two binary numbers, then it will affect the subtraction of next stages. Therefore, the subtraction with borrow are performed by a full subtractor.

The block diagram and circuit diagram of a full-subtractor is shown in Figure-1.





## Operation of Full Subtractor

Now, let us understand the operation of the full subtractor. Full subtractor performs its operation to find the difference of two binary numbers according to the rules of binary subtraction, which are as follows –

In the case of full subtractor, the 1s and 0s for the output variables (difference and borrow) are determined from the subtraction of  $A - B - b_{in}$ .

From the logic circuit diagram of the full subtractor, it is clear that the difference bit (d) is obtained by the XOR operation of the two inputs A, B, and  $b_{in}$ , and the output borrow bit (b) is obtained by NOT, AND, and OR operations of variable A, B, and  $b_{in}$ .



## Truth Table of Full-Subtractor

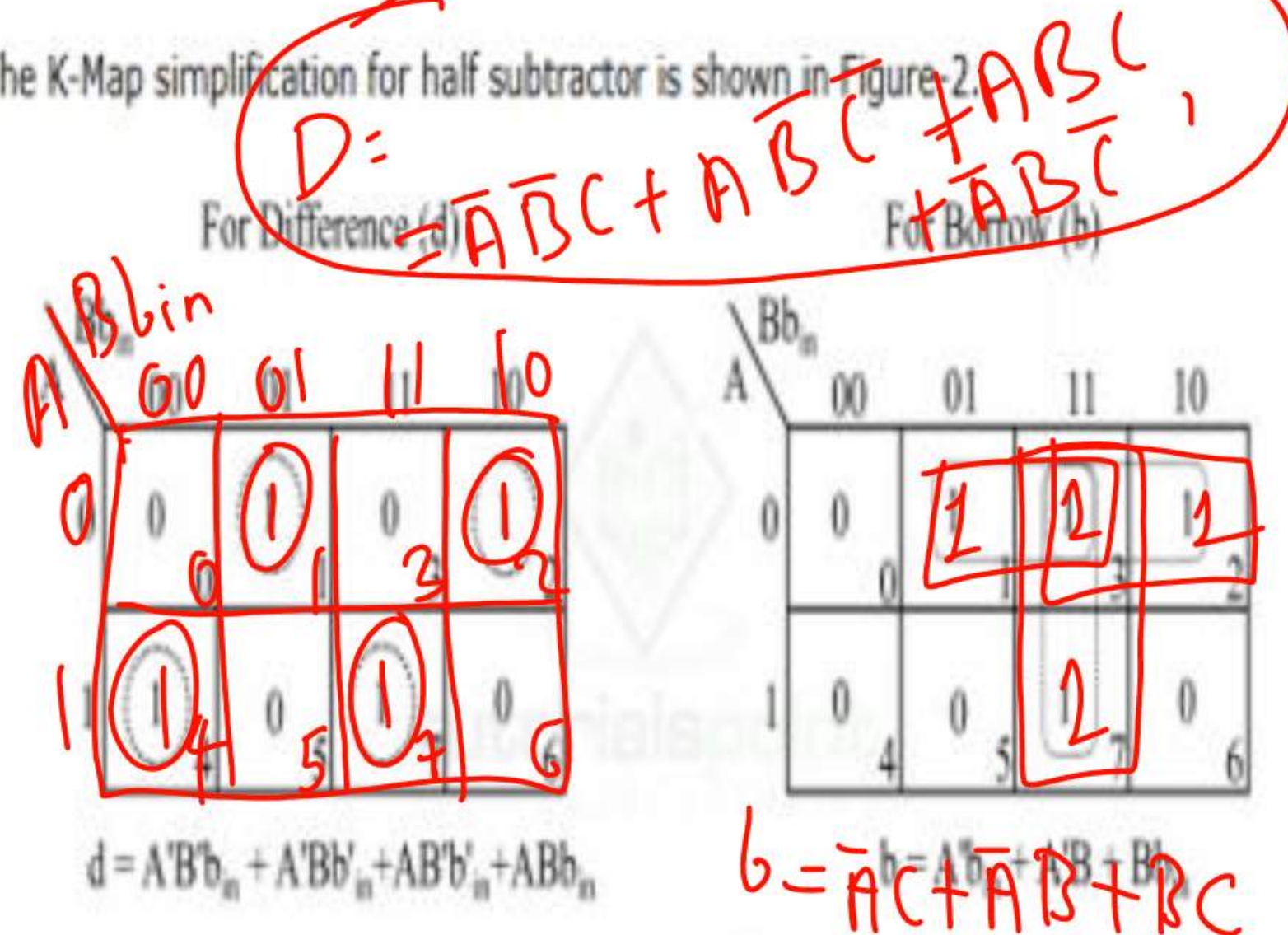
The truth table is one that gives relationship between input and output of a logic circuit. The following is the truth table of the full-subtractor –

| Inputs |   |                 | Outputs        |                           |
|--------|---|-----------------|----------------|---------------------------|
| A      | B | B <sub>in</sub> | D (Difference) | B <sub>out</sub> (Borrow) |
| 0      | 0 | 0               | 0              | 0                         |
| 0      | 0 | 1               | 1              | 1                         |
| 0      | 1 | 0               | 1              | 1                         |
| 0      | 1 | 1               | 0              | 1                         |
| 1      | 0 | 0               | 1              | 0                         |
| 1      | 0 | 1               | 0              | 0                         |
| 1      | 1 | 0               | 0              | 0                         |
| 1      | 1 | 1               | 1              | 1                         |

## K-Map for Full Subtractor

We can use the K-Map (or Karnaugh Map), a method for simplifying Boolean algebra, to determine equations of the difference bit (d) and the output borrow bit (b).

The K-Map simplification for half subtractor is shown in Figure 2.



$$Y = A \oplus B \oplus C$$

$$D = \overline{A} \overline{B} \overline{C} + A \overline{B} \overline{C} + A B \overline{C} + \overline{A} B \overline{C} + \overline{A} B C + A \overline{B} C + A B C$$

3 Terms

$$A \overline{B}$$

$$\overline{A} \overline{B} \overline{C} + \overline{A} \overline{B} C$$

$$\overline{A}$$

$$\overline{A}$$

$$\overline{A} \overline{B} \overline{C}$$

$$A$$

$$\overline{A}$$

$$\overline{A} B C$$

$$\overline{A} B C$$

$$\overline{A} \overline{B} \overline{C} + \overline{A} \overline{B} C + \overline{A} B \overline{C} + \overline{A} B C$$

~~XOR = XNOR~~

XOR

| A | B | C |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

| A | B |
|---|---|
| 0 | 0 |
| 0 | 1 |
| 1 | 0 |
| 1 | 1 |

XNOR

$$D = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$$

$$= C(\bar{A}\bar{B} + AB) + \bar{C}(\bar{A}B + A\bar{B})$$

00    11

$$= C(\overline{A\bar{B} + \bar{A}B}) + \bar{C}(\bar{A}B + A\bar{B})$$

$$= C(\overline{A \oplus B}) + \bar{C}(A \oplus B)$$

$$= A \oplus B \oplus C$$

| C |
|---|
| 1 |
| 0 |
| 0 |
| 1 |



# Characteristic Equations of Full Subtractor

The characteristic equations of the full subtractor, i.e. equations of the difference (d) and borrow output (b) are obtained by following the rules of binary subtraction. These equations are given below –

The difference (d) of the full subtractor is the XOR of A, B, and  $b_{in}$ . Therefore,

$$\text{Difference, } d = A \oplus B \oplus b_{in} = A'B'b_{in} + AB'b'_{in} + A'Bb'_{in} + ABb_{in}$$

The borrow (b) of the full subtractor is given by,

**From the logic circuit diagram and k-map –**

$$\text{Borrow, } b = A'B + (A \oplus B)'b_{in}$$

**From Truth Table**

$$\text{Borrow, } b = A'B'b_{in} + A'Bb'_{in} + A'Bb_{in} + ABb_{in}$$



# Applications of Full Subtractor

The following are some important applications of full subtractor –

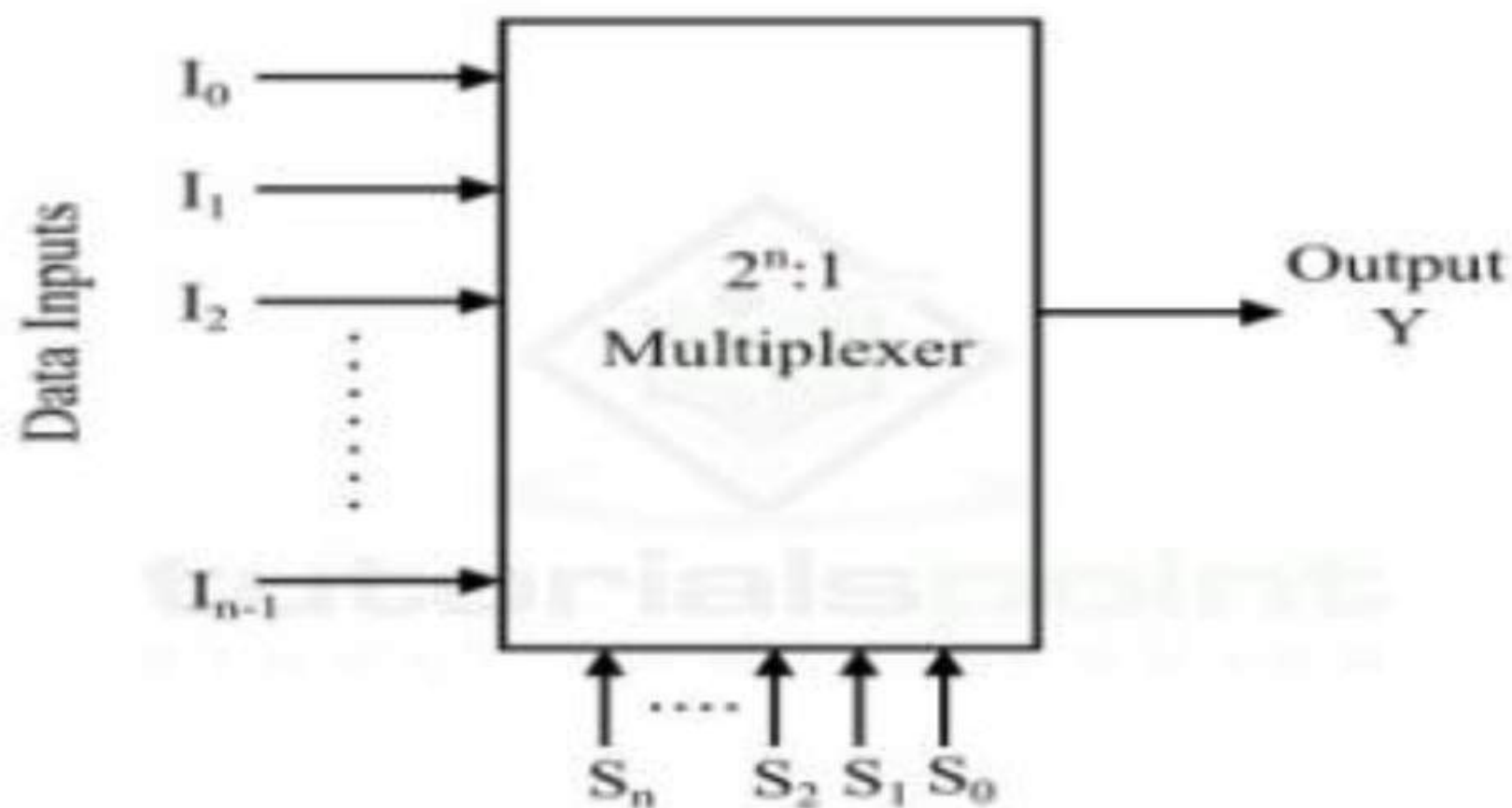
- Full subtractors are used in ALU (Arithmetic Logic Unit) in computers CPUs.
- Full subtractors are extensively used to perform arithmetical operations like subtraction in electronic calculators and many other digital devices.
- Full subtractors are used in different microcontrollers for arithmetic subtraction.
- They are used in timers and program counters (PC).
- Full subtractors are also used in processors to compute addresses, tables, etc.
- Full subtractors are also used in DSP (Digital Signal Processing) and networking based systems.



# What is a Multiplexer?

As already mentioned, a **multiplexer**, also referred to as **MUX**, is a combination logic circuit that is designed to accept multiple input signals and transfer only one of them through the output line. In simple words, a multiplexer is a digital logic device that selects one-out-of-N ( $N = 2^n$ ) input data sources and transmits the selected data to a single output line.

The multiplexer is also called **data selector** as it selects one from several. The block diagram of a typical  $2^n:1$  multiplexer is shown in Figure 1.





In the case of multiplexer, the selection of desired data input to flow through the output line is controlled with the help of **SELECT lines**. In the block diagram of mux in Figure 1,  $I_0, I_1, \dots, I_{n-1}$ , i.e.,  $(2^n)$  are the input lines, and "**n**" be the select lines. These select lines will determine which input is to be routed to the output.

Hence, the multiplexer works as a multi-position switch whose operation is controlled by digital signals. These digital control signals are applied to the select lines to determine which data input will be switched to the output line.



# Function of Multiplexer

Multiplexer is a digital logic device which is used to perform **multiplexing** of data. Where, multiplexing simply means sharing of data. Technically, when a particular data is selected from multiple input data sources and transmitted the selected data to a single output channel, it is called **multiplexing**.

There are two types of multiplexing namely, **frequency multiplexing** and **time multiplexing**.

When multiple devices are connected to a single transmission line in a system. At any point of time, only one device is using the line to transmit data, then this is called time multiplexing. On the other hand, when multiple devices share a common line to transmit data but at different frequencies, it is called frequency multiplexing.



**1:- 2x1 Multiplexer**

**2:- 4x1 Multiplexer**

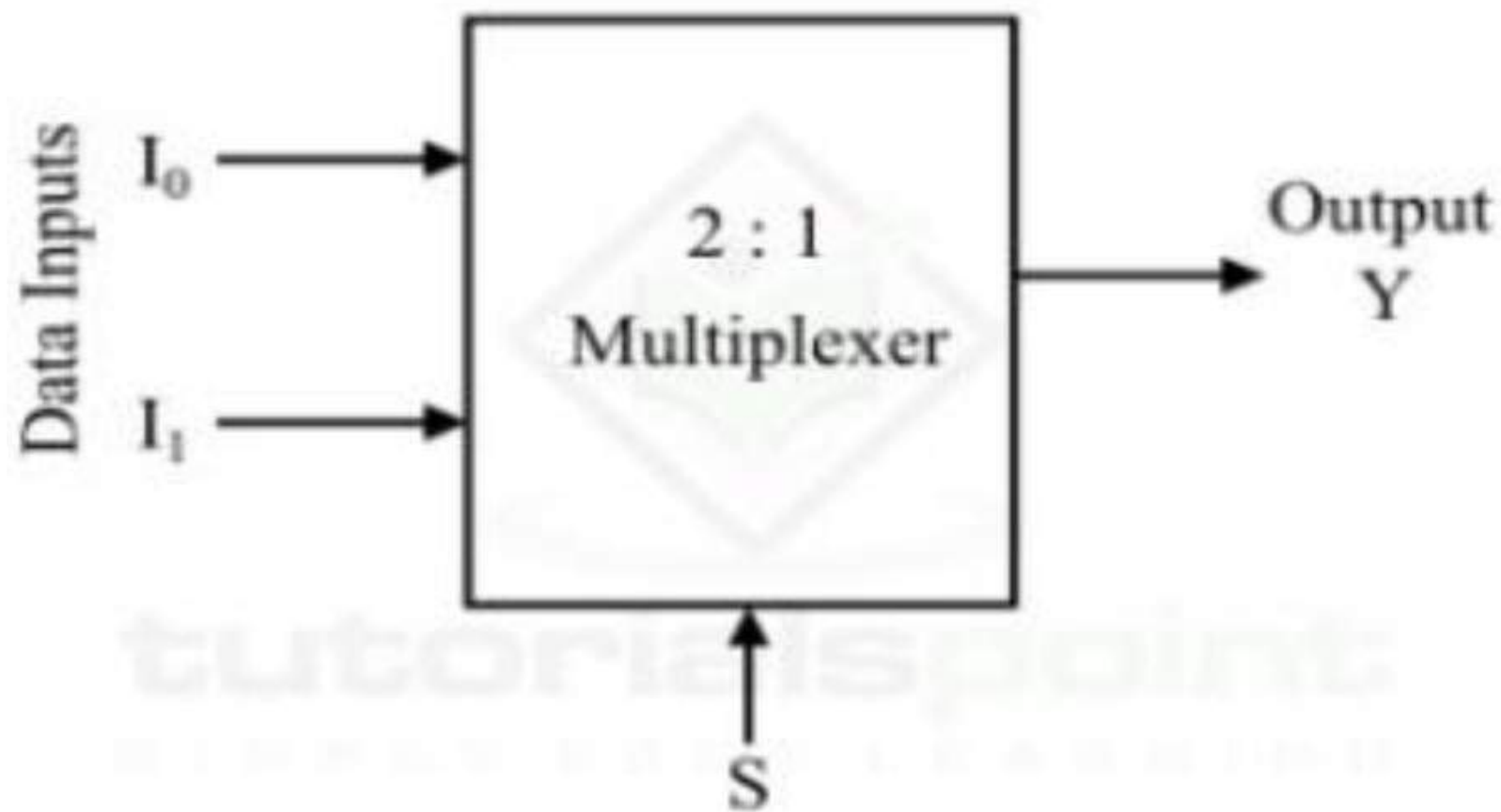
**3:- 8x1 Multiplexer**

**4:- 16x1 Multiplexer**



### 2x1 Multiplexer

The block diagram of a 2x1 multiplexer is shown in Figure 2. The 2x1 multiplexer is basic two input multiplexer which has two data input lines designated as  $I_0$  and  $I_1$ , one data select line denoted by  $S$  and one output line denoted by  $Y$ . The 2x1 mux is used to connect two 1-bit data sources to a common designation.





In the 2×1 multiplexer, the logic level of the digital signal applied to the select line S determines which data input will pass through the output line. The operation of the 2×1 multiplexer can be understood from the following truth table.

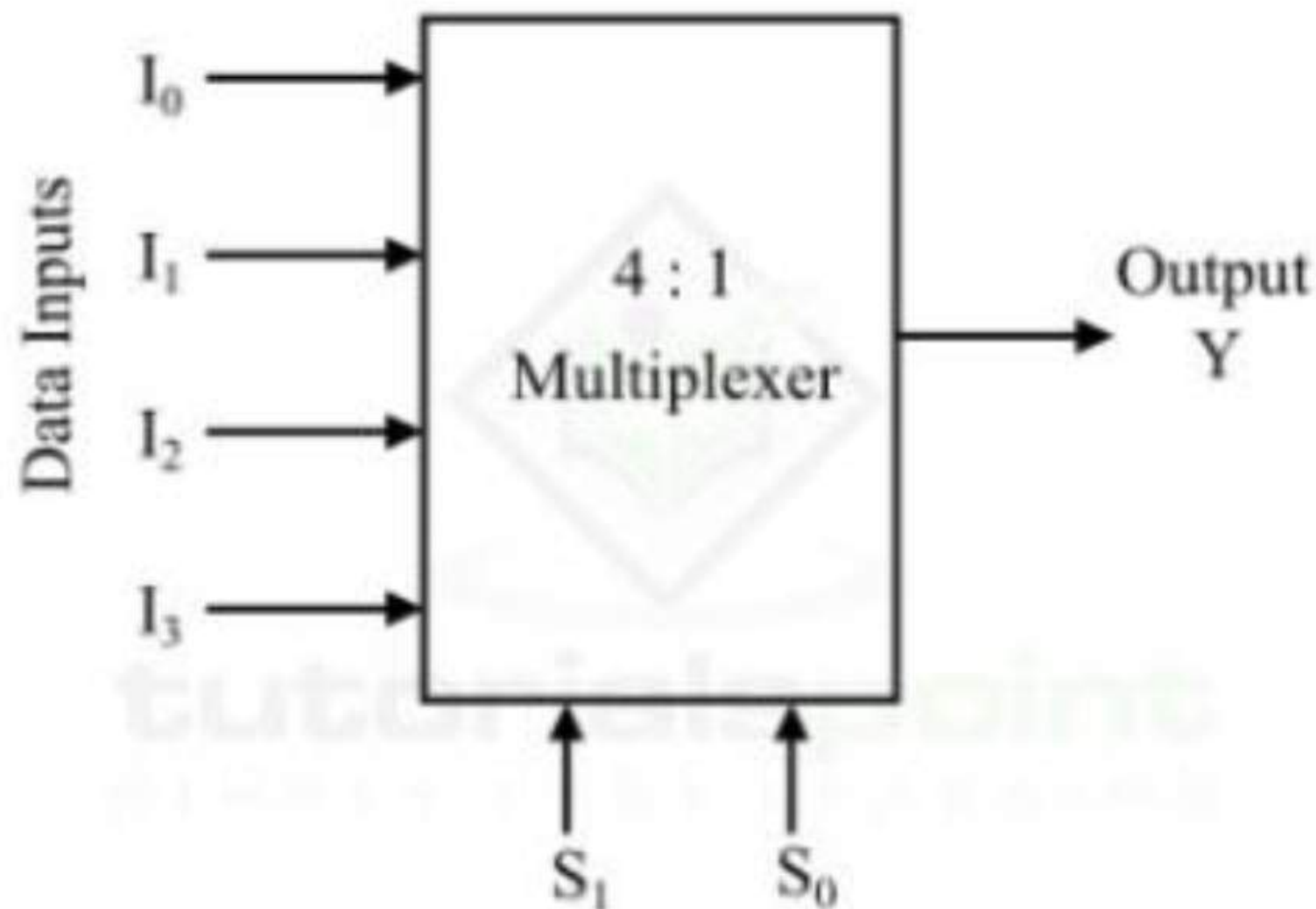
| Select Line (S) | Output (Y) |
|-----------------|------------|
| 0               | $I_0$      |
| 1               | $I_1$      |

$$Y = S' \cdot I_0 + S \cdot I_1$$



### 4x1 Multiplexer

4x1 Multiplexer has four data inputs  $I_3$ ,  $I_2$ ,  $I_1$  &  $I_0$ , two selection lines  $s_1$  &  $s_0$  and one output  $Y$ . The **block diagram** of 4x1 Multiplexer is shown in the following figure.





One of these 4 inputs will be connected to the output based on the combination of inputs present at these two selection lines. **Truth table** of 4×1 Multiplexer is shown below.

| Selection Lines |       | Output |
|-----------------|-------|--------|
| $S_1$           | $S_0$ | $Y$    |
| 0               | 0     | $I_0$  |
| 0               | 1     | $I_1$  |
| 1               | 0     | $I_2$  |
| 1               | 1     | $I_3$  |

From Truth table, we can directly write the **Boolean function** for output,  $Y$  as

$$Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$$

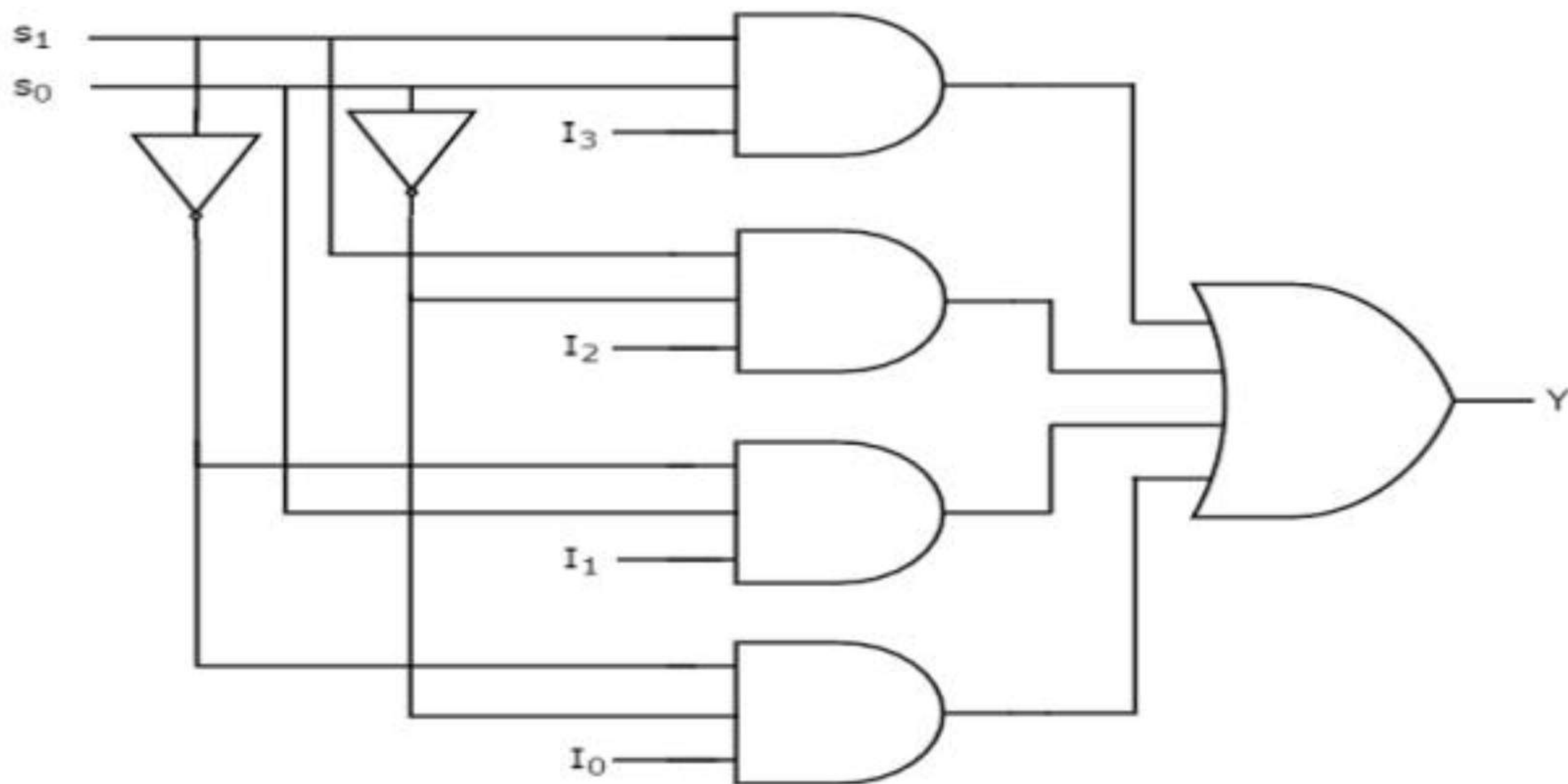


## Topic : 4x1 Multiplexer



TEACHING  
WALLAH

We can implement this Boolean function using Inverters, AND gates & OR gate. The **circuit diagram** of 4x1 multiplexer is shown in the following figure.





# 8x1 Multiplexer

In this section, let us implement 8x1 Multiplexer using 4x1 Multiplexers and 2x1 Multiplexer. We know that 4x1 Multiplexer has 4 data inputs, 2 selection lines and one output. Whereas, 8x1 Multiplexer has 8 data inputs, 3 selection lines and one output.

So, we require two **4x1 Multiplexers** in first stage in order to get the 8 data inputs. Since, each 4x1 Multiplexer produces one output, we require a **2x1 Multiplexer** in second stage by considering the outputs of first stage as inputs and to produce the final output.



## Topic : 8x1 Multiplexer



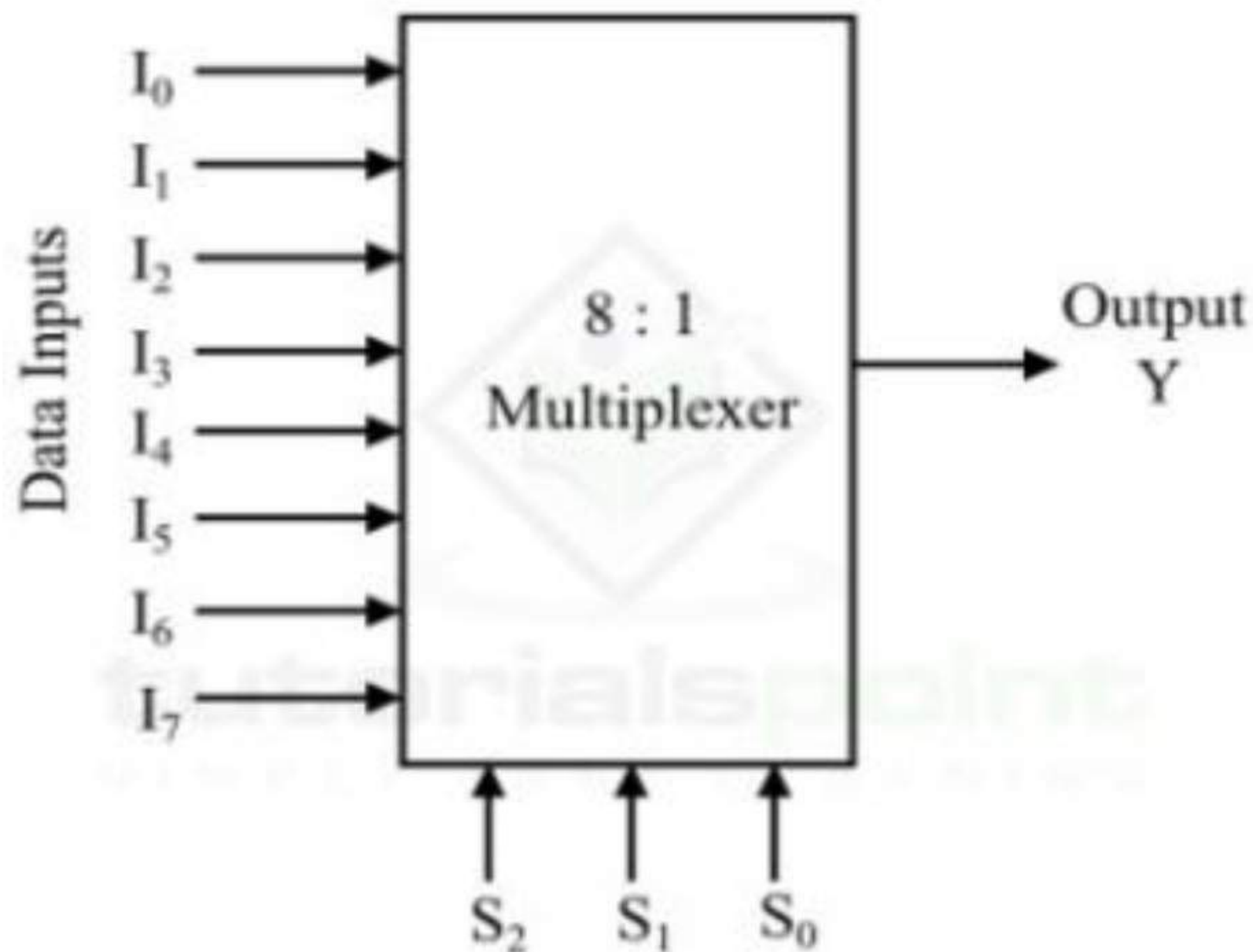
TEACHING  
WALLAH

Let the 8×1 Multiplexer has eight data inputs  $I_7$  to  $I_0$ , three selection lines  $s_2$ ,  $s_1$  &  $s_0$  and one output  $Y$ . The **Truth table** of 8×1 Multiplexer is shown below.

| Selection Inputs |       |       | Output |
|------------------|-------|-------|--------|
| $S_2$            | $S_1$ | $S_0$ | $Y$    |
| 0                | 0     | 0     | $I_0$  |
| 0                | 0     | 1     | $I_1$  |
| 0                | 1     | 0     | $I_2$  |
| 0                | 1     | 1     | $I_3$  |
| 1                | 0     | 0     | $I_4$  |
| 1                | 0     | 1     | $I_5$  |
| 1                | 1     | 0     | $I_6$  |
| 1                | 1     | 1     | $I_7$  |



We can implement  $8 \times 1$  Multiplexer using lower order Multiplexers easily by considering the above Truth table. The **block diagram** of  $8 \times 1$  Multiplexer is shown in the following figure.

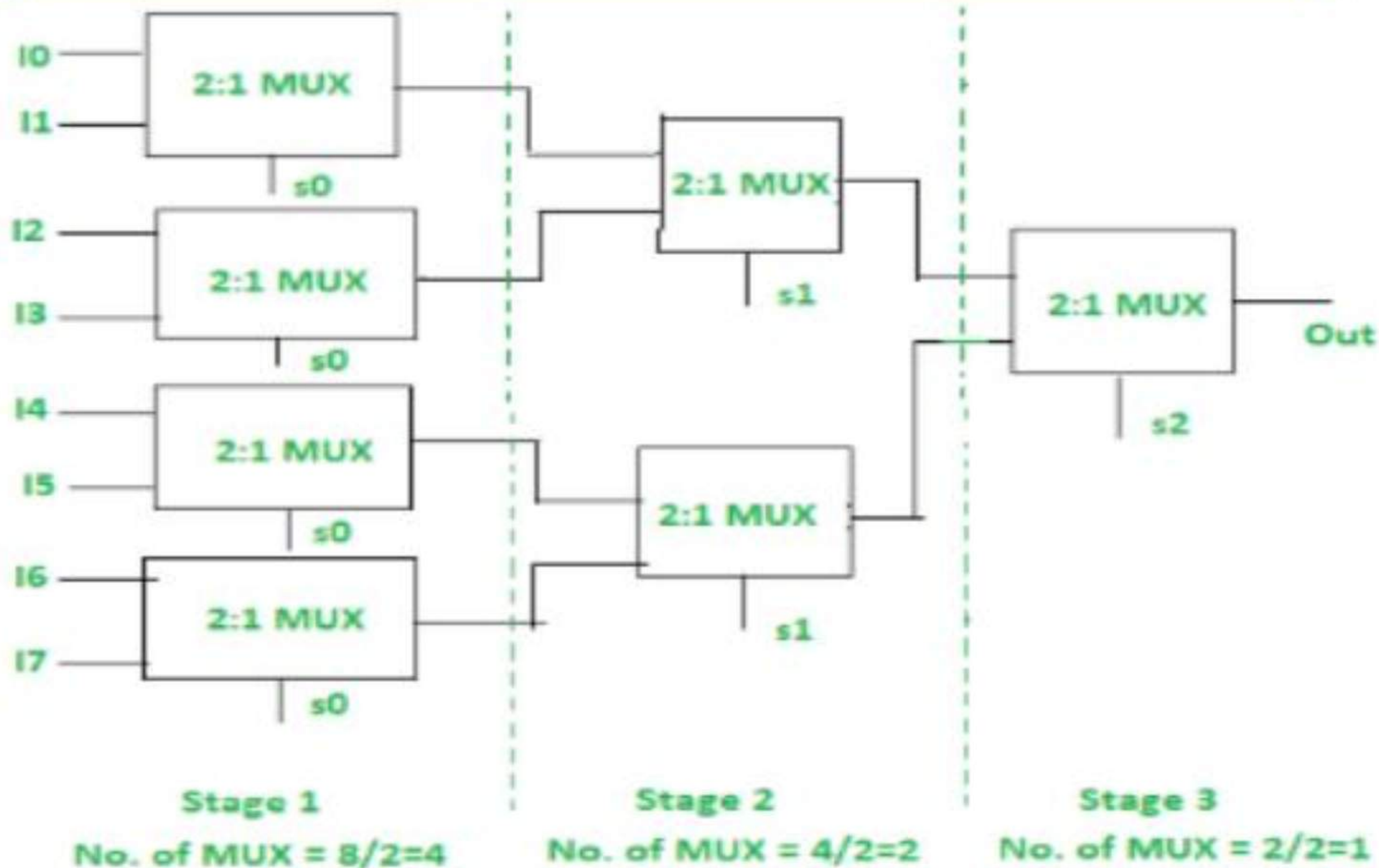




# Topic : 8x1 Multiplexer by using 2 x1 mux



TEACHING  
WALLAH

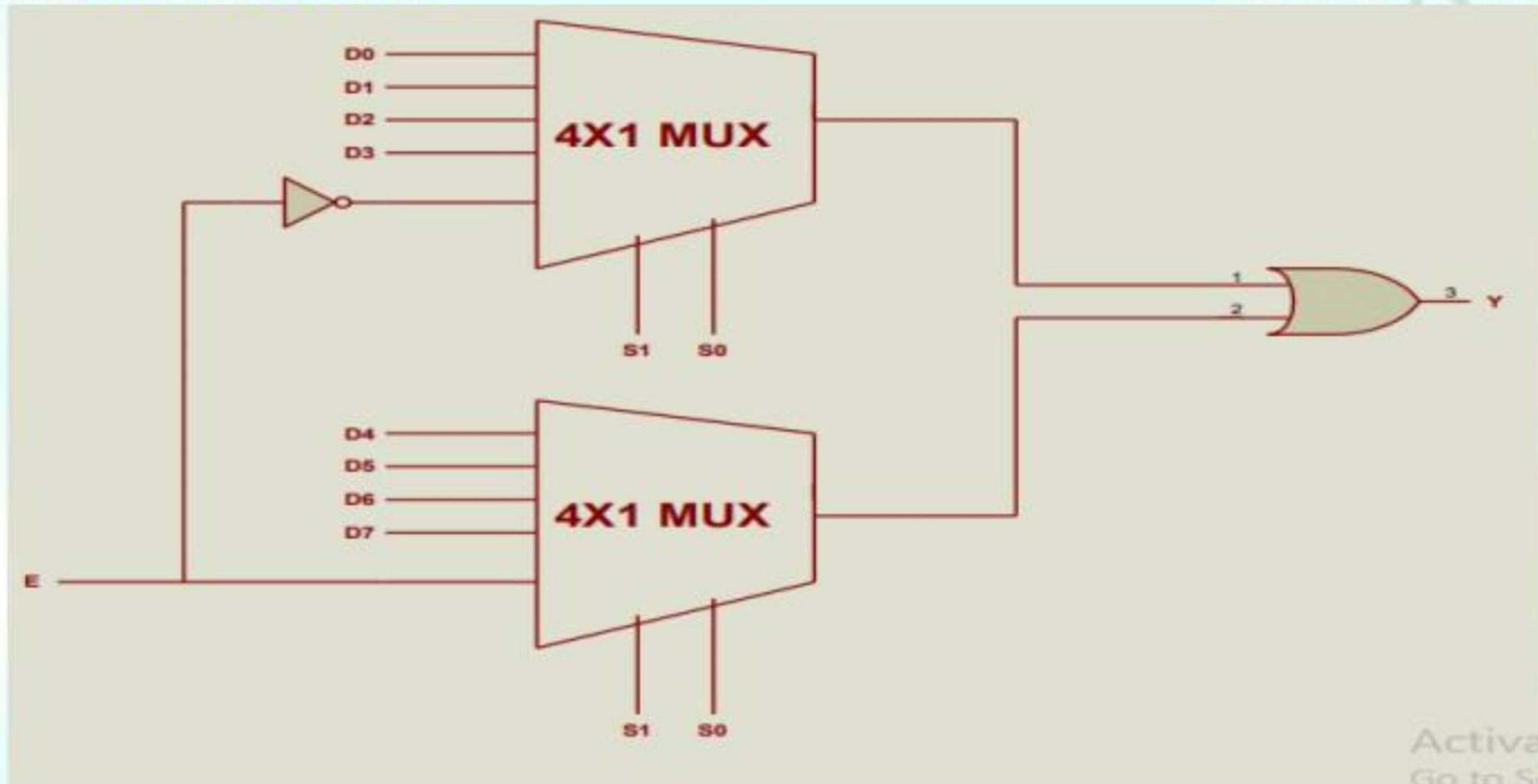




# Topic : 8x1 Multiplexer by using 4x1 Mux- special case (Enable System)



TEACHING  
WALLAH



**THANK YOU**